

Lambda Functions.

What Lambda actually is

It's a **serverless compute service** where you:

- Upload code (Python, Node.js, etc.)
- AWS runs it **only when triggered**
- You don't manage EC2, OS, scaling, nothing

Simple analogy (no fluff)

- EC2 → You rent a machine and manage everything
- Lambda → You just give a task, AWS runs it and disappears

How it works (real flow)

1. You write a function
2. You upload it to Lambda
3. Something triggers it:
 - API call
 - File upload to S3
 - Timer (cron job)
4. Lambda runs → finishes → stops

I have created an Lambda Function To delete the EC2 and EBS.

I have created an EC2 and EBS Volume And Attached to that instance.

EC2

Dashboard
AWS Global View
Events

▼ Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances

Successfully attached volume vol-0bb780508201aa0ec to instance i-054dbedb7736c86cf.

Volumes (2) Info

Choose filter set

Search

Size	IOPS	Throughput	Snapshot ID	Source volume ID	Created	Availability Zone	Volume state
20 GiB	3000	125	-	-	2026/03/31 21:48 GMT+5...	aps2-az1 (ap-south-2a)	In-use
8 GiB	3000	125	snap-030bfb1...	-	2026/03/31 21:43 GMT+5...	aps2-az1 (ap-south-2a)	In-use

EC2 > Instances

EC2

Dashboard
AWS Global View
Events

▼ Instances
Instances
Instance Types
Launch Templates

Instances (1) Info

Choose filter set

Find Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
linux	i-054dbedb7736c86cf	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-2a	ec2-98-130

Then I have Created An Lambda Functions.

Lambda > Functions > Create function

Create function Info

Choose one of the following options to create your function.

☒ Author from scratch
Start with a simple Hello World example.

☐ Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.

☐ Container image
Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

myfunction

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.14

Architecture

Choose the instruction set architecture you want for your function code.

☐ arm64

☒ x86_64

Permissions

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

► Change default execution role

Python 3.14

Architecture

Info

Choose the instruction set architecture you want for your function code.

arm64

x86_64

Permissions

Info

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ Change default execution role

Execution role

To access other AWS services and resources your function needs an IAM role. Choose a role with the right permissions for your function.

Create default role

Use another role

ec2my

Create new role

[View role details in IAM](#)

▼ Additional configurations

Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

Networking

VPC

Info

Connect your function to a VPC to access private resources during invocation.

Enable

Functions (1)

Last fetched 31/03/2026, 21:57:39

Actions

Create function

Search by attributes or search by keyword

Function name	Description	Package type	Runtime	Last modified
myfunction	-	Zip	Python 3.14	33 seconds ago

Then I have deployed the code.

CodeTestMonitorConfigurationAliasesVersions

Executing function: succeeded (logs)

Details

```
{
  "statusCode": 400,
  "body": "Volume is not in 'available' state. Current state: in-use"
}
```

Summary

Code SHA-256

R6fL2f5y9xHPT04NojRY2hz04DIAxWgwXm4xSMnwRQ=

Execution time

13 seconds ago

Function version

\$LATEST

Request ID

0b32ad92-2e92-46d4-91ab-749fb9d15c0a

Duration

2832.80 ms

Billed duration

2833 ms

Resources configured

Max memory used

The Testing Phase.

TEST EVENTS [SELECTED: NEWTEST]

Create new test event

Private saved events

newtest

ENVIRONMENT VARIABLES

Lambda Deployed

Amazon Q

PROBLEMS

OUTPUT

CODE REFERENCE LOG

TERMINAL

Execution Results

Status: Succeeded

Test Event Name: newTest

Response:

```
{
  "statusCode": 200,
  "body": "Volume vol-0bb780508201aa0ec detached and deleted successfully"
}
```

The area below shows the last 4 KB of the execution log.

Code properties

Info

Package size

652 byte

SHA256 hash

McaGpYHaxbQEIBO+mCu3ppBwphtLuxiQN4PAxsFJHY=

Last modified

39 seconds ago

Runtime settings

Info

Edit

Edit runtime management configuration

Command to Delete and Instance And Detach From the Instance From the EBS Volume.

```
import boto3

ec2 = boto3.client('ec2')

def lambda_handler(event, context):

    instance_id = 'i-xxxxxxxxxxxx' # change this

    # 1. Get all attached volumes

    response = ec2.describe_instances(InstanceIds=[instance_id])

    volumes = []

    for r in response['Reservations']:

        for i in r['Instances']:

            for b in i.get('BlockDeviceMappings', []):

                volumes.append(b['Ebs']['VolumeId'])

    # 2. Terminate instance

    ec2.terminate_instances(InstanceIds=[instance_id])

    # Wait until instance is terminated

    waiter = ec2.get_waiter('instance_terminated')

    waiter.wait(InstanceIds=[instance_id])

    # 3. Delete volumes (only non-auto deleted ones will remain)

    for vol in volumes:

        try:

            ec2.delete_volume(VolumeId=vol)
```

```
    print(f"Deleted volume: {vol}")
except Exception as e:
    print(f"Skipping {vol}: {str(e)}")
return "EC2 and EBS deletion complete"
```

And then deleting the Detached Volume.

```
import boto3
ec2 = boto3.client('ec2')
def lambda_handler(event, context):
    volumes = ec2.describe_volumes(
        Filters=[{'Name': 'status', 'Values': ['available']}]
    )
    for vol in volumes['Volumes']:
        vid = vol['VolumeId']
        try:
            ec2.delete_volume(VolumeId=vid)
            print(f"Deleted: {vid}")
        except Exception as e:
            print(f"Error deleting {vid}: {str(e)}")

    return "Cleanup complete"
```

stopped Instance:

EC2 > Instances

Dashboard
AWS Global View
Events

▼ Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans

Instances (1/1) Info

Last updated less than a minute ago

Connect Instance state Actions Launch instances

Saved filter sets
Choose filter set

Find Instance by attribute or tag (case-sensitive)

☒	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☒	linux	i-054dbedb7736c86cf	Stopping	t3.micro	-	View alarms +	ap-south-2a	ec2-18-61-6...

Then deleted the volume.

EC2

Dashboard
AWS Global View
Events

▼ Instances
Instances
Instance Types

Volumes (1) Info

Last updated less than a minute ago

Recycle Bin Actions Create volume

Saved filter sets
Choose filter set

Search

☐	Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Source volume ID	Created
☐		vol-0ea8b340592ba48b9	gp3	8 GiB	3000	125	snap-030bfb1...	-	2026/03/...

Then deleted the Elastic ip

EC2 > Elastic IP addresses

Dashboard
AWS Global View
Events

▼ Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances

Elastic IP addresses Info

Find elastic IP addresses by attribute or tag

☐	Name	Allocated IPv4 address	Type	Allocation ID	Reverse DNS record	Assoc...
No Elastic IP addresses found in this Region						